



DOKUZ EYLUL UNIVERSITY

ENGINEERING FACULTY

DEPARTMENT OF COMPUTER ENGINEERING

CME2210 OBJECT ORIENTED ANALYSIS AND DESIGN PHASE 2 REPORT

Beauty System — CenGIRL

by

Şengül Akkaya 2023510121

Arzu Hamdioğlu 2023510054

Zeliha Erdoğan 2023510145

Lecturer

Prof. Dr. Semih Utku

IZMIR

April 2026

Table of Contents

1. **Introduction** 3
2. **System Architecture** 3
   2.1 Architectural Layers 3
   2.2 Component Interaction 4
   2.3 Design Patterns Applied 4
3. **Class Diagram** 6
   3.1 Key Classes and Relationships 6
   3.2 Class Diagram Visualization 7
4. **Sequence Diagrams** 8
   4.1 Create Appointment 8
   4.2 Cancel Appointment 8
   4.3 Complete Service and Make Payment 9
   4.4 Pay Salary 10
5. **Statechart Diagrams** 12
   5.1 Appointment Lifecycle 12
   5.2 Payment Lifecycle 13
6. **Activity Diagrams** 15
   6.1 User Login and Authentication 15
   6.2 Appointment Booking Process 16
   6.3 Payment Processing 17
7. **Overall System Design Description** 19
8. **Conclusion** 20

1. Introduction

This document constitutes the Phase 2 Report for the CenGIRL – Beauty Salon Management System, developed as part of the CME2210 Object Oriented Analysis and Design course at Dokuz Eylul University. Building upon the requirements analysis performed in Phase 1, this phase focuses on transforming those requirements into a structured and detailed system design.

The report covers the overall system architecture, a comprehensive class diagram, sequence diagrams for key use cases, statechart diagrams illustrating critical state transitions, and activity diagrams depicting the major process flows within the system.

2. System Architecture

CenGIRL employs a strict Layered (N-Tier) architecture that enforces a clear separation of concerns. Every module in the system belongs to one of four well-defined layers, ensuring that business logic remains independent of user-interface concerns and data-access mechanisms.

2.1 Architectural Layers

Presentation Layer

The Presentation Layer is responsible for all console-based user interactions. It reads input from actors (Customer, Employee, Manager) and delegates every request to the Application Layer. This layer contains no business logic; it solely handles formatting, input parsing, and displaying results or error messages.

Application / Service Layer

The Application Layer coordinates use-case workflows by orchestrating domain objects and infrastructure services. Classes such as AppointmentService, PaymentService, and NotificationService reside here. This layer enforces authorization rules, validates inputs before passing them to the Domain Layer, and ensures transactional consistency.

Domain Layer

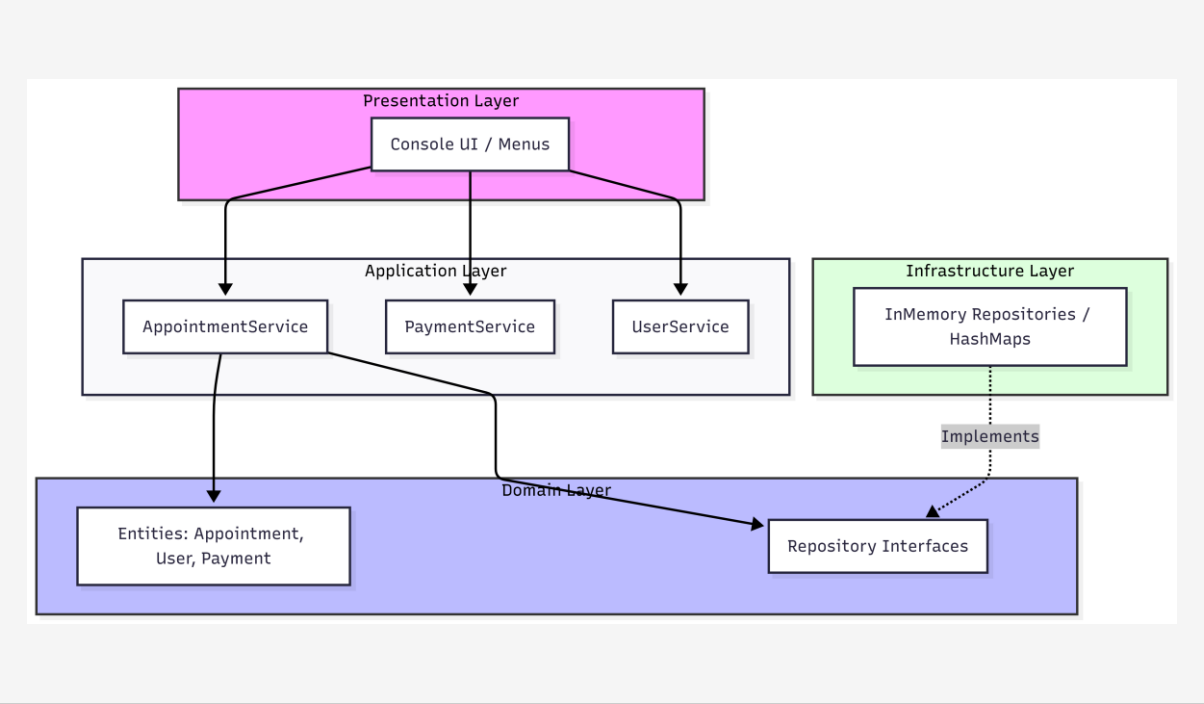
The Domain Layer is the core of the system and contains the rich business model. It houses all domain entities (User, Customer, Employee, Manager, Appointment, Service, Payment), value objects, and domain-specific rules. All state transitions and business invariants are enforced here using strict encapsulation and access modifiers.

Infrastructure Layer

The Infrastructure Layer provides in-memory repository implementations for all domain entities. Repositories use Java collection types (HashMap, HashSet) to ensure O(1) average-case lookup performance. The layer also includes helper utilities such as the IdGenerator and DateTimeUtil classes.

2.2 Component Interaction

The following architecture diagram illustrates how the four layers interact. The arrows indicate the direction of dependency; upper layers depend on lower layers, but never the reverse, preserving the Dependency Inversion Principle.



2.3 Design Patterns Applied

Several design patterns are integrated throughout the architecture to achieve extensibility and maintainability:

- Strategy Pattern: Used in the PaymentService to support multiple payment methods (cash, credit card, digital wallet) without modifying core logic.
- Factory Method Pattern: Applied in the UserFactory to create Customer, Employee, and Manager instances without coupling the service layer to concrete classes.
- Observer Pattern: Used to implement the NotificationService; appointment creation and cancellation events automatically notify all registered observers (customer and employee).
- Repository Pattern: All data access is abstracted behind repository interfaces, enabling the in-memory implementations to be swapped for database-backed ones without any change to the Application or Domain layers.

3. Class Diagram

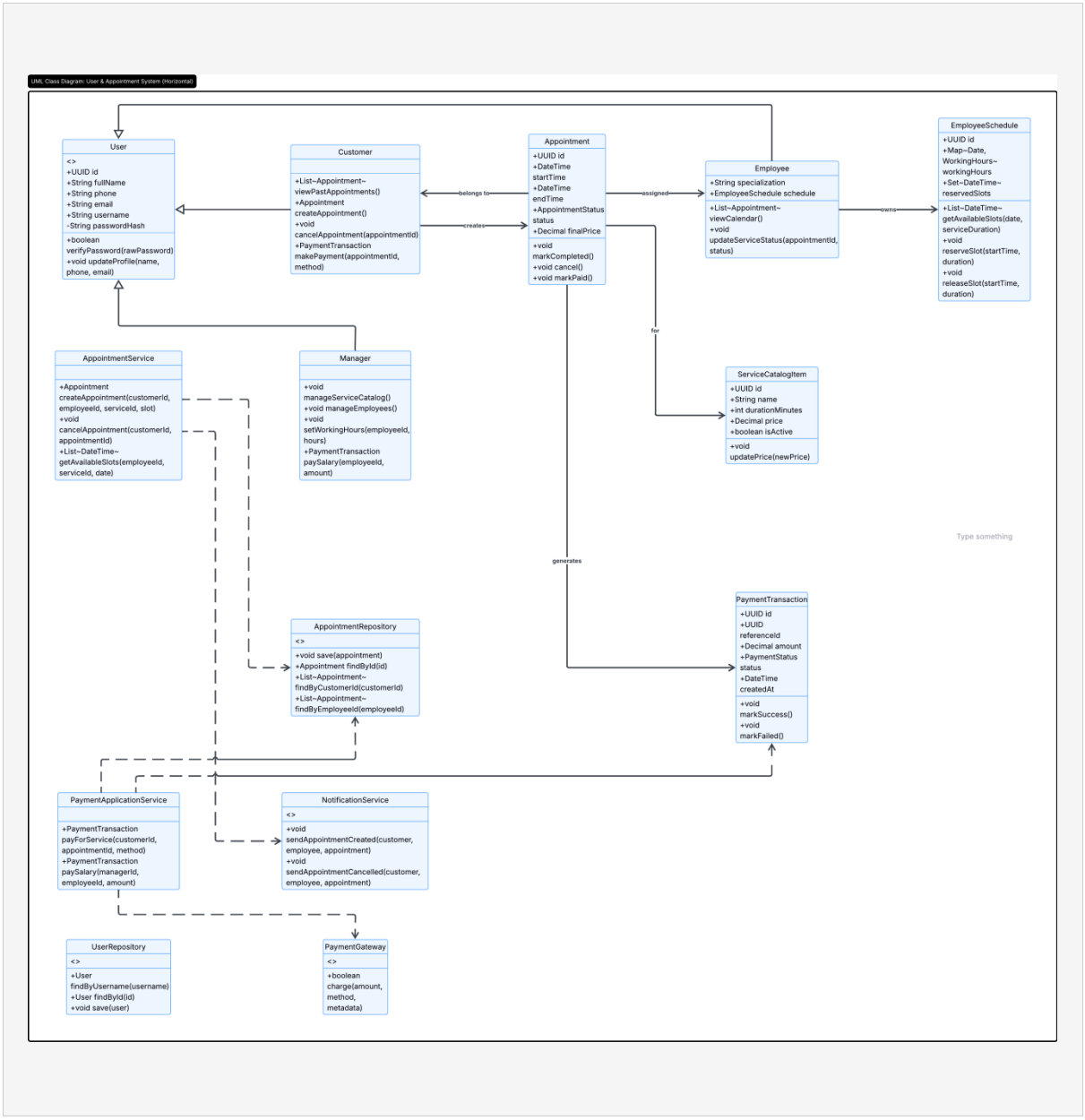
The class diagram below presents the complete object model of the CenGIRL system. It captures all major domain entities, their attributes, methods, and the relationships between them (inheritance, association, aggregation, and dependency).

3.1 Key Classes and Relationships

The following table summarises the primary classes, their responsibilities, and their relationships to other classes:

Class	Responsibility	Key Relationships
User (abstract)	Base class for all actors; holds credentials and profile data.	Inherited by Customer, Employee, Manager
Customer	Manages appointments, views services, makes payments.	Aggregates List<Appointment>
Employee	Manages calendar, updates service statuses.	Aggregates Calendar; associated with List<Appointment>
Manager	Controls service catalog, employee records, payroll.	Aggregates List<Employee>, List<Service>
Appointment	Central domain entity; holds service, time slot, and status.	Associated with Customer, Employee, Service, Payment
Service	Represents a beauty service (name, duration, price).	Referenced by Appointment
Payment	Tracks financial transactions for appointments and salaries.	Associated with Appointment or Employee
AppointmentService	Use-case orchestration for appointment CRUD.	Depends on AppointmentRepository, NotificationService
PaymentService	Processes customer payments and manager salary payments.	Uses Strategy pattern for payment methods
NotificationService	Sends notifications on appointment events.	Observer of Appointment lifecycle

3.2 Class Diagram

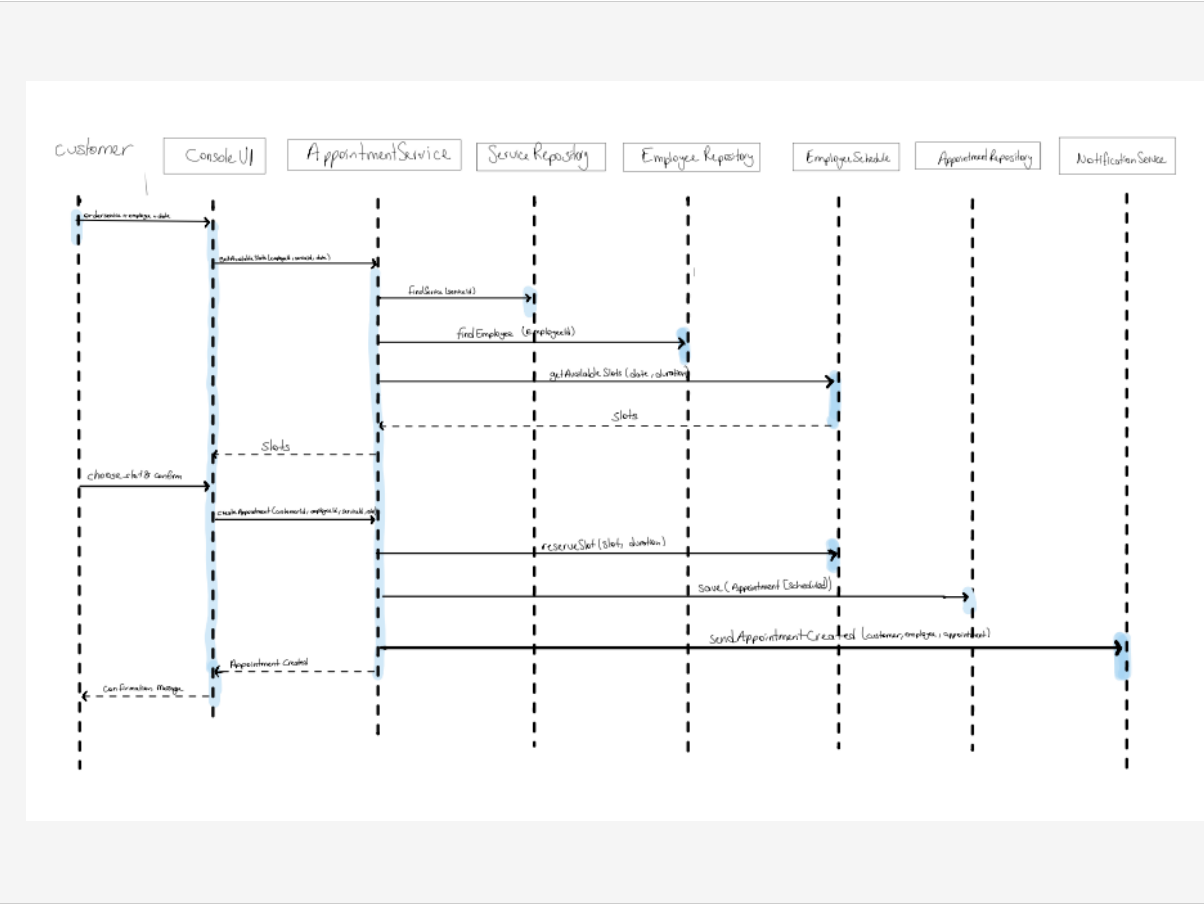

UML Class Diagram: User & Appointment System (Horizontal)
User
<>
+UUID id
+String fullName
+String phone
+String email
+String username
-String passwordHash
+boolean verifyPassword(rawPassword)
+void updateProfile(name, phone, email)
Customer
+List<Appointment> viewPastAppointments()
+Appointment createAppointment()
+void cancelAppointment(appointmentId)
+PaymentTransaction makePayment(appointmentId, method)
belongs to
creates
Appointment
+UUID id
+DateTime startTime
+DateTime endTime
+AppointmentStatus status
+Decimal finalPrice
+void markCompleted()
+void cancel()
+void markPaid()
assigned
Employee
+String specialization
+EmployeeSchedule schedule
+List<Appointment> viewCalendar()
+void updateServiceStatus(appointmentId, status)
owns
EmployeeSchedule
+UUID id
+Map<Date, WorkingHours> workingHours
+Set<DateTime> reservedSlots
+List<DateTime> getAvailableSlots(date, serviceDuration)
+void reserveSlot(startTime, duration)
+void releaseSlot(startTime, duration)
for
AppointmentService
+Appointment createAppointment(customerId, employeeId, serviceId, slot)
+void cancelAppointment(customerId, appointmentId)
+List<DateTime> getAvailableSlots(employeeId, serviceId, date)
Manager
+void manageServiceCatalog()
+void manageEmployees()
+void setWorkingHours(employeeId, hours)
+PaymentTransaction paySalary(employeeId, amount)
ServiceCatalogItem
+UUID id
+String name
+int durationMinutes
+Decimal price
+boolean isActive
+void updatePrice(newPrice)
Type something
generates
AppointmentRepository
<>
+void save(appointment)
+Appointment findById(id)
+List<Appointment> findByCustomerId(customerId)
+List<Appointment> findByEmployeeId(employeeId)
PaymentTransaction
+UUID id
+UUID referenceId
+Decimal amount
+PaymentStatus status
+DateTime createdAt
+void markSuccess()
+void markFailed()
PaymentApplicationService
+PaymentTransaction payForService(customerId, appointmentId, method)
+PaymentTransaction paySalary(managerId, employeeId, amount)
NotificationService
<>
+void sendAppointmentCreated(customer, employee, appointment)
+void sendAppointmentCancelled(customer, employee, appointment)
UserRepository
<>
+User findByUsername(username)
+User findById(id)
+void save(user)
PaymentGateway
<>
+boolean charge(amount, method, metadata)

4. Sequence Diagrams

Sequence diagrams illustrate the interactions between objects over time for specific use cases. The four diagrams below correspond to the most critical workflows in the system.

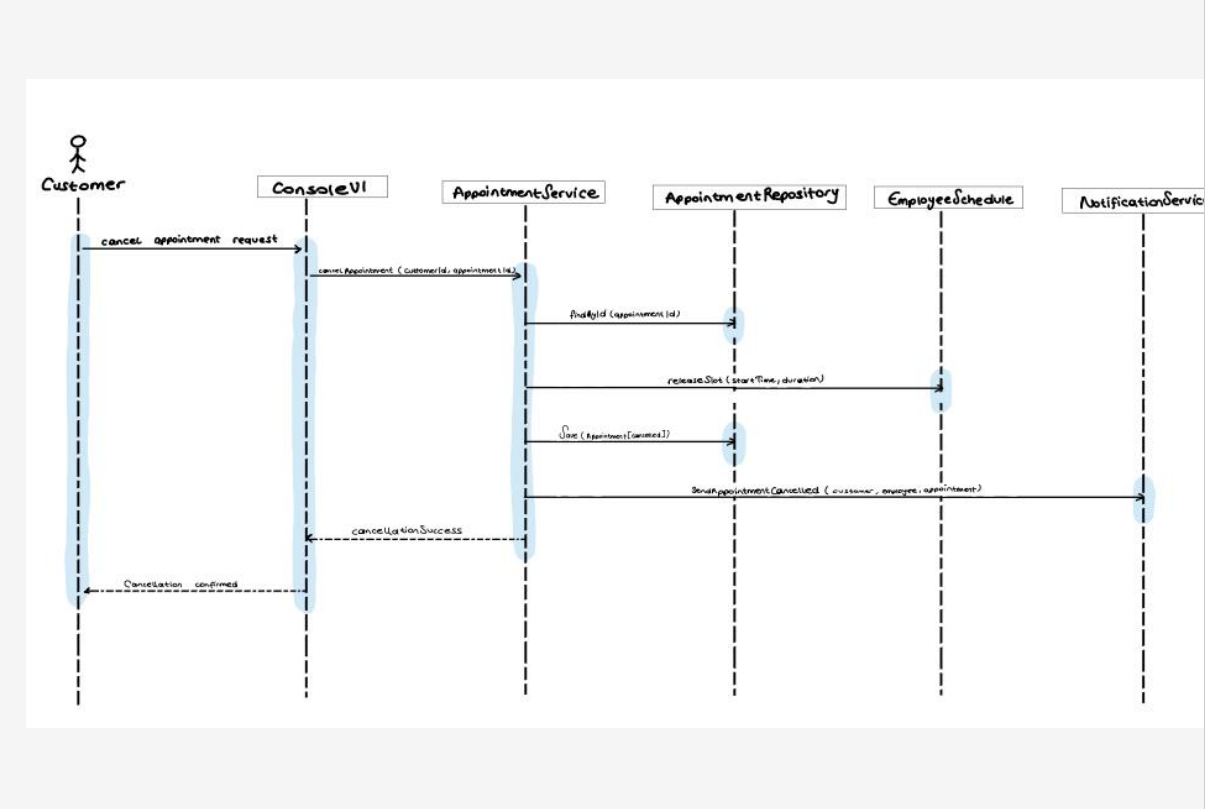
4.1 Create Appointment

This sequence captures the end-to-end flow when a customer creates a new appointment. The customer selects a service and employee via the Presentation Layer, which delegates to the AppointmentService. The service validates availability through the AppointmentRepository, persists the new Appointment entity, and triggers the NotificationService to dispatch a booking confirmation to both the customer and the assigned employee.



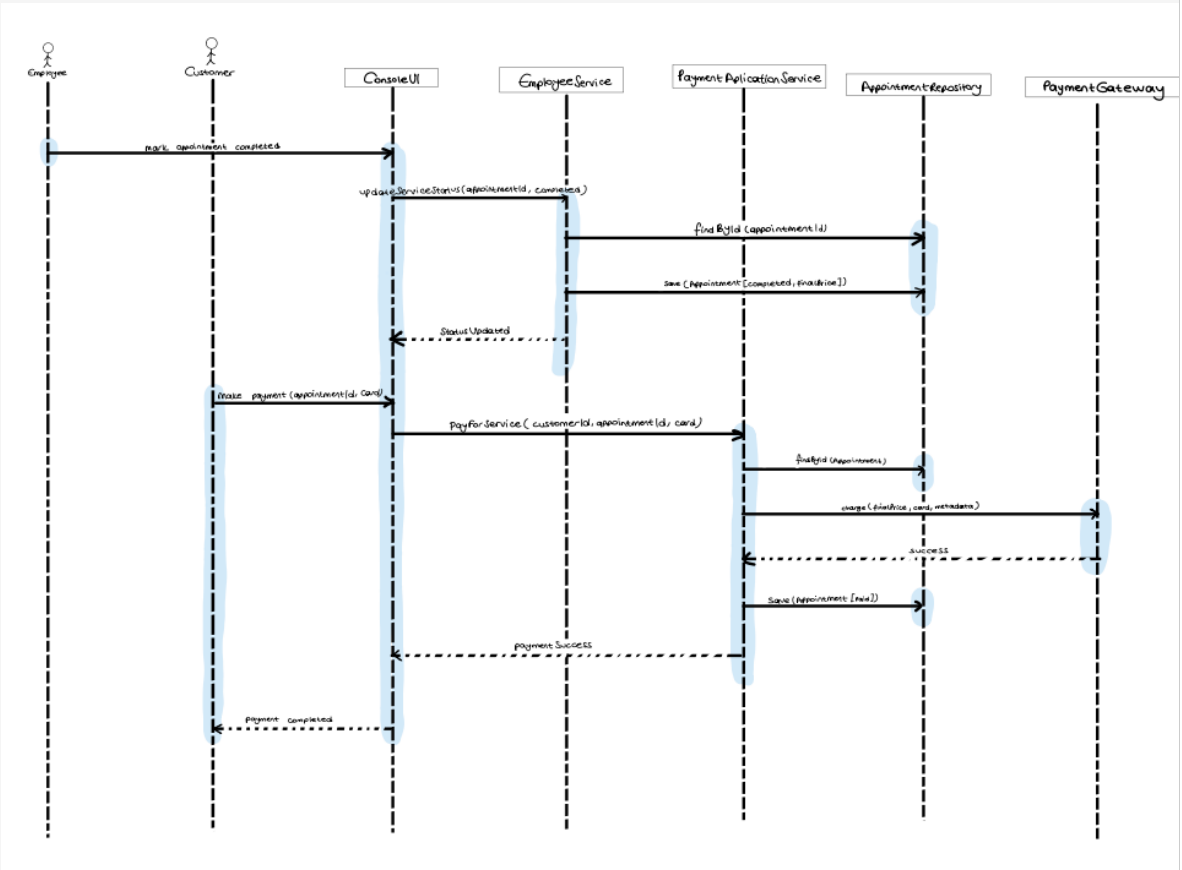
4.2 Cancel Appointment

When a customer requests a cancellation, the AppointmentService first verifies that the appointment is in an active (Scheduled) state. It then updates the appointment status to Cancelled, releases the time slot in the employee's Calendar, and notifies both parties via the NotificationService.



4.3 Complete Service and Make Payment

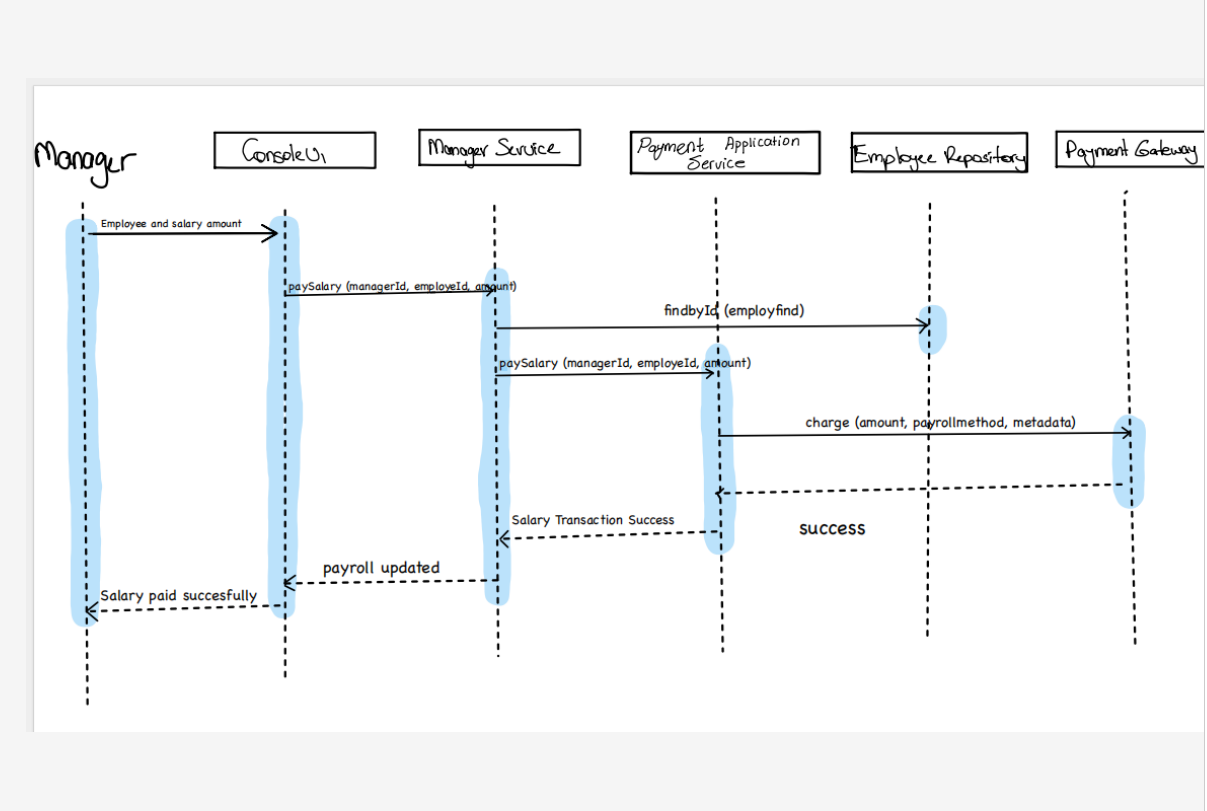
This two-part sequence begins with the Employee marking a scheduled appointment as Completed, which triggers the creation of a pending Payment record. Subsequently, the Customer accesses the payment interface, and the PaymentService processes the transaction using the selected payment strategy. On success, the appointment status is updated to Paid.


Employee
Customer
ConsoleUI
EmployeeService
PaymentApplicationService
AppointmentRepository
PaymentGateway
mark appointment completed
updateServiceStatus(appointmentId, completed)
findById(appointmentId)
save(Appointment[completed, finalPrice])
StatusUpdated
makePayment(appointmentId, card)
payForService(customerId, appointmentId, card)
findById(Appointment)
charge(finalPrice, card, metadata)
success
save(Appointment[paid])
paymentSuccess
paymentCompleted

]

4.4 Pay Salary

The Manager selects an employee from the payroll module and confirms the monthly salary amount. The PaymentService processes the transaction and updates the Employee's financial record to Paid for the current period. A confirmation message is returned to the manager.



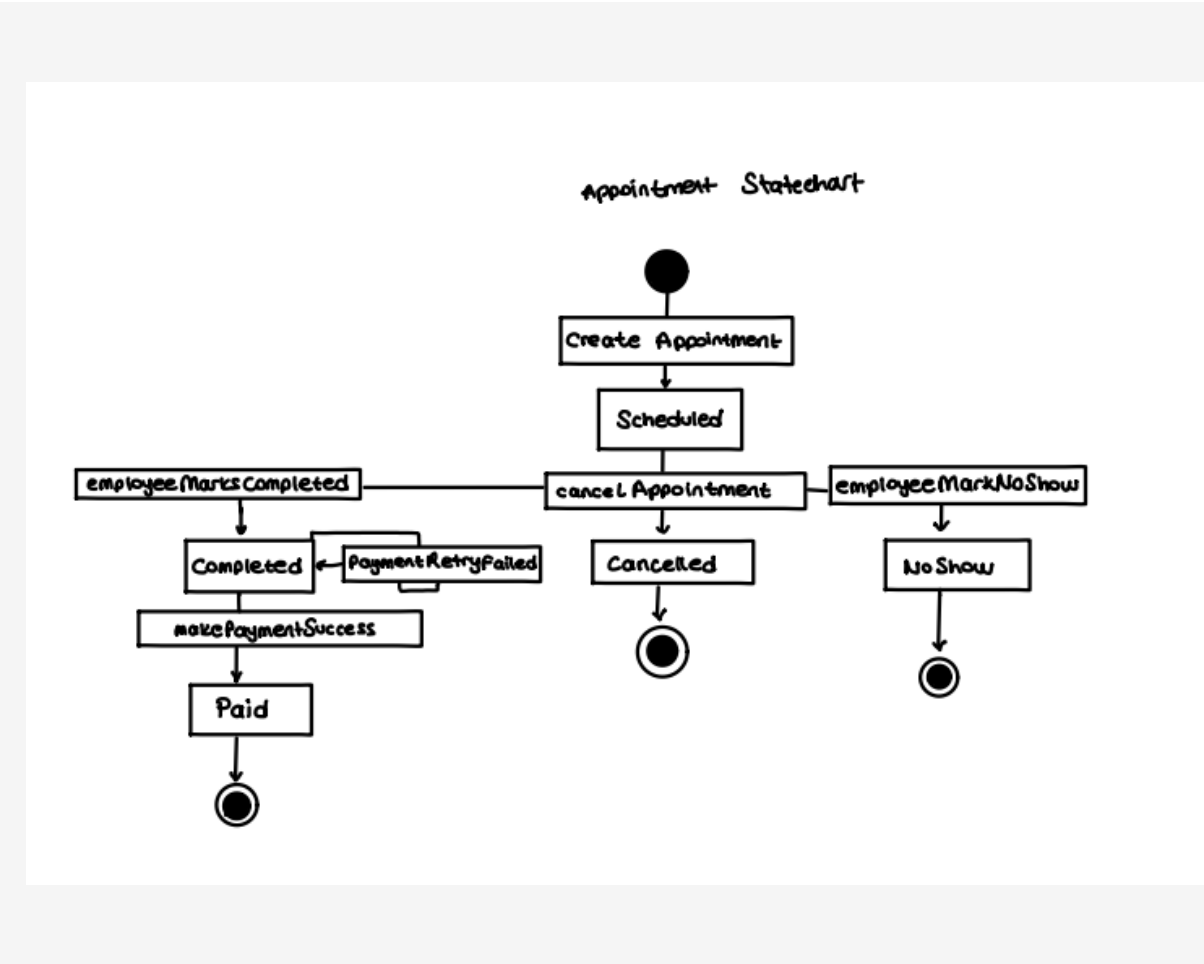
5. Statechart Diagrams

Statechart diagrams model the lifecycle of critical domain objects by capturing all possible states and the transitions between them. Two objects with rich state behaviour are selected: the Appointment entity and the Payment entity.

5.1 Appointment Lifecycle

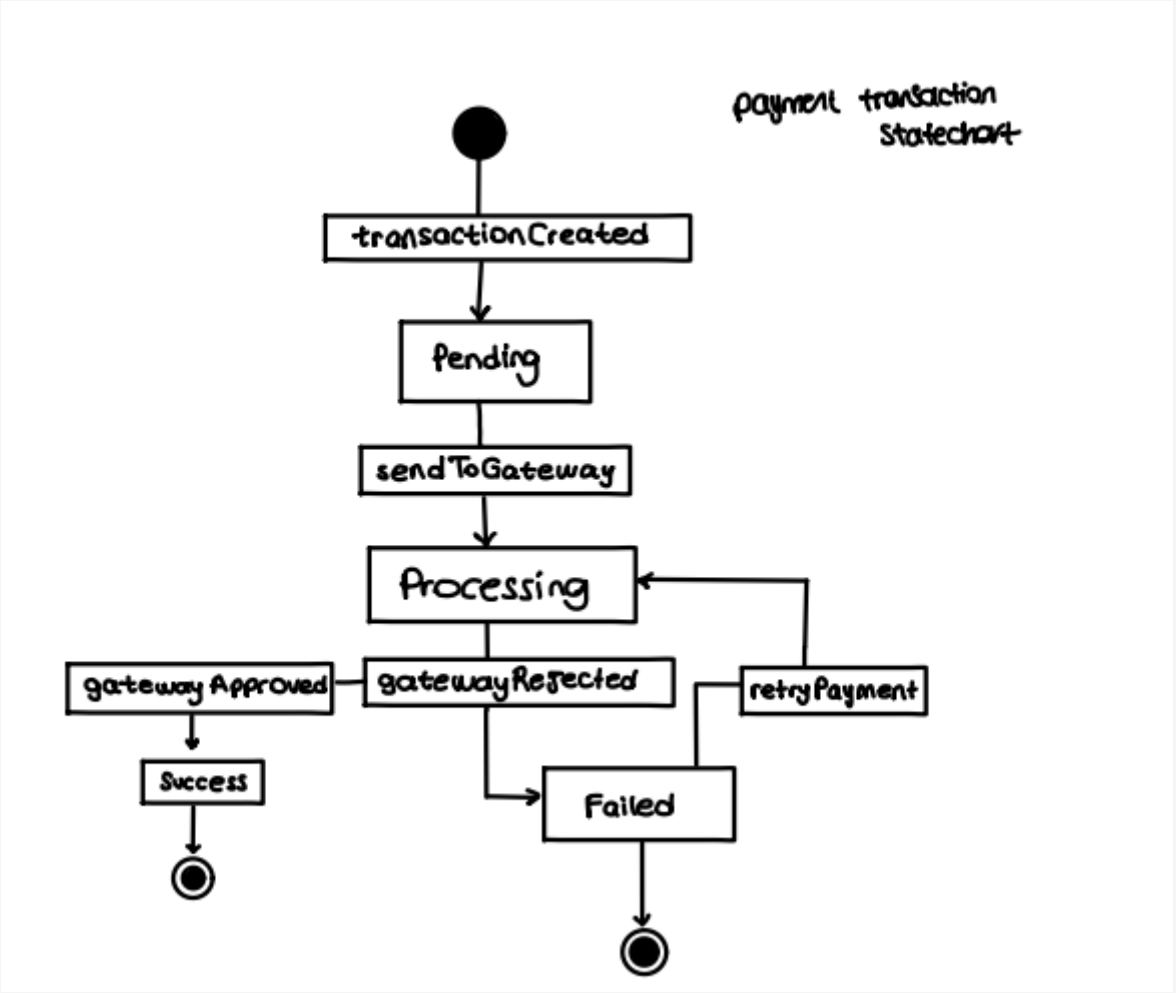
An Appointment begins in the Scheduled state immediately upon creation. It can transition to Completed when the employee marks the service as done, to Cancelled when the customer requests cancellation, or to NoShow if the customer does not attend. From Completed, the appointment moves to Paid once the customer's payment is successfully processed.

Current State	Trigger	Next State
(initial)	Customer creates appointment	**Scheduled**
Scheduled	Employee marks service done	**Completed**
Scheduled	Customer cancels	**Cancelled**
Scheduled	Customer does not attend	**NoShow**
Completed	Customer payment successful	**Paid**



5.2 Payment Lifecycle

A Payment record is created in a Pending state. If the transaction is processed successfully by the PaymentService, the record transitions to Completed. If the transaction fails (e.g., due to insufficient funds or a system error), it transitions to Failed, which allows for a retry. A Completed payment may be refunded, resulting in a Refunded terminal state.

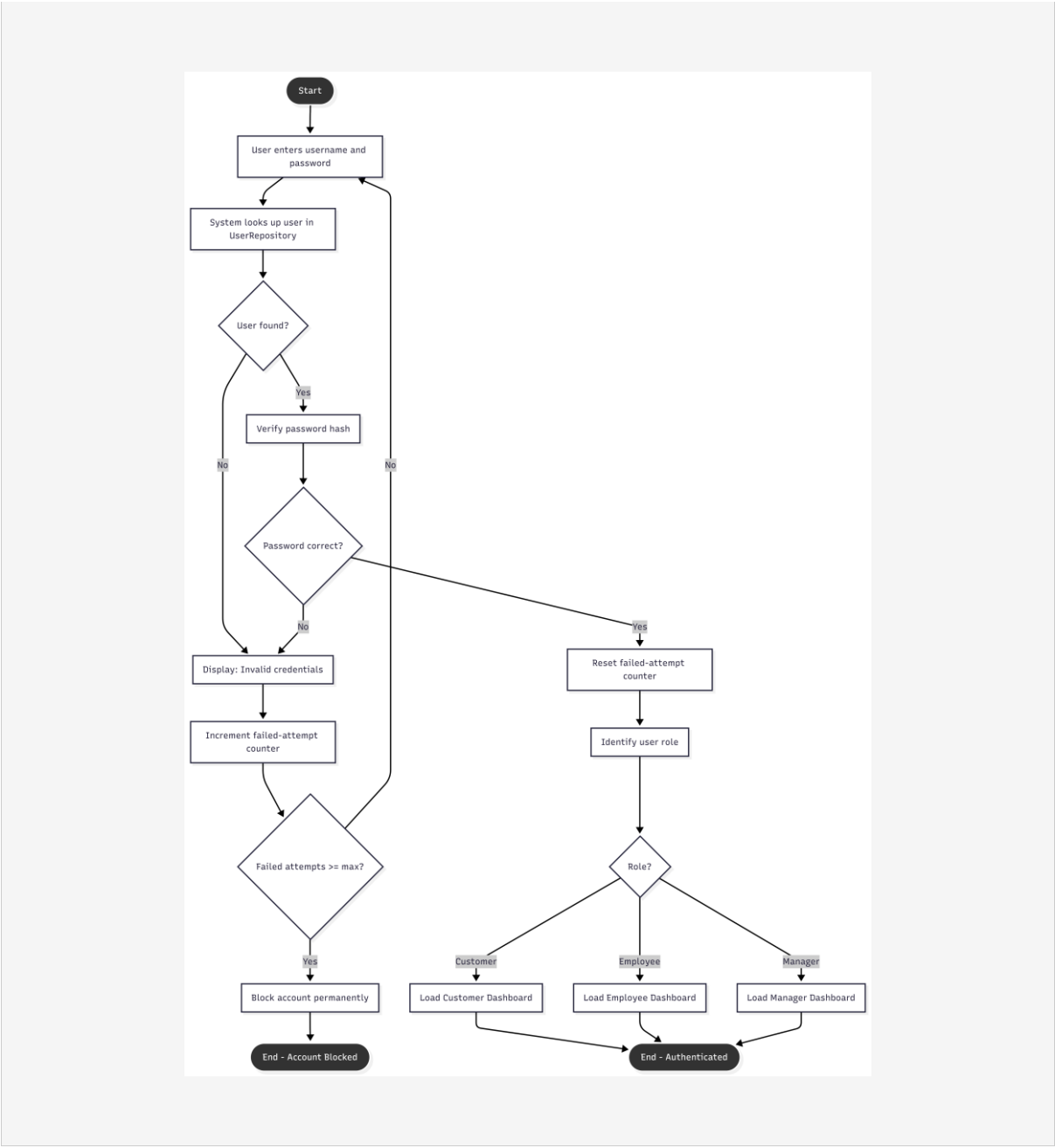

payment transaction
statechart
transactionCreated
Pending
sendToGateway
Processing
gatewayApproved
gatewayRejected
retryPayment
Success
Failed

6. Activity Diagrams

Activity diagrams model the sequential and concurrent flow of activities within a process, including decision points and parallel paths. Three key workflows are represented below.

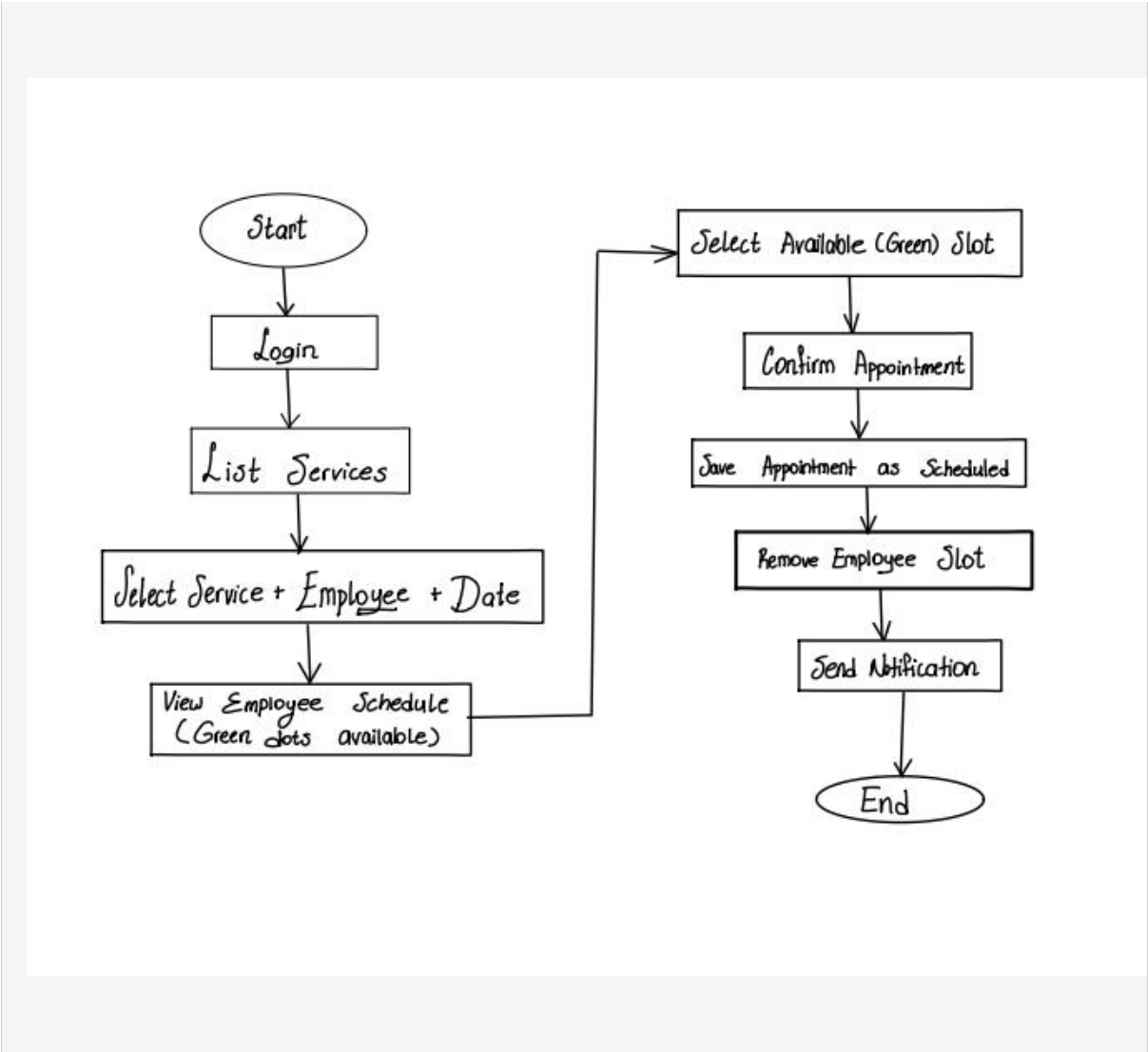
6.1 User Login and Authentication

This activity diagram covers the authentication flow common to all actors. The user enters their credentials; the system verifies them against the in-memory repository. If verification fails, the system increments the failed-attempt counter. After a configurable number of consecutive failures, the account is temporarily locked. On successful verification, the system loads the role-specific dashboard.



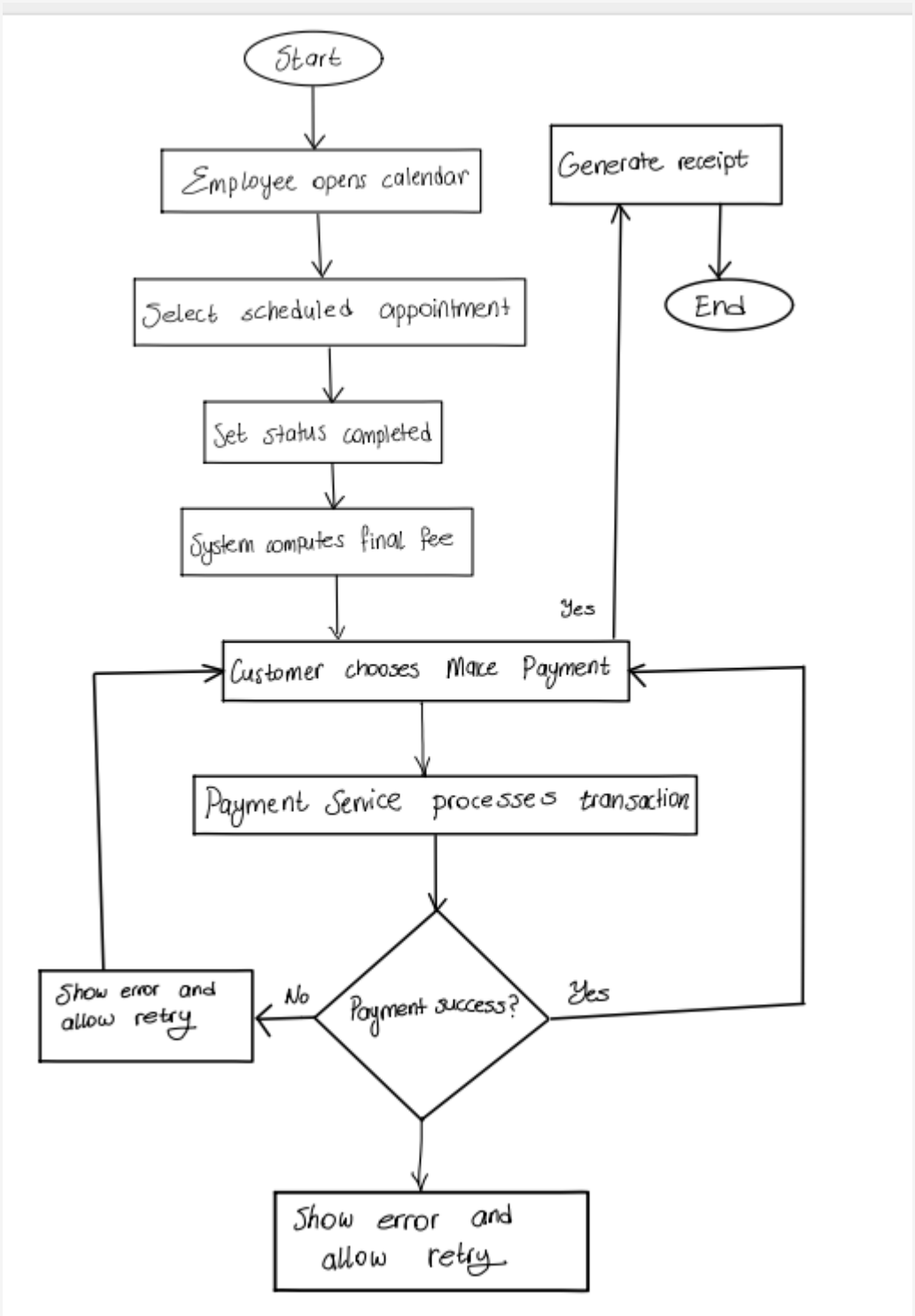
6.2 Appointment Booking Process

The booking workflow starts when an authenticated customer navigates to the appointment creation screen. The customer selects a service and an employee; the system fetches available time slots from the employee's Calendar. After the customer confirms the booking, the system checks for concurrent conflicts (double-booking guard), persists the appointment, updates the employee's calendar, and dispatches confirmation notifications to both parties.



6.3 Payment Processing

When a customer initiates payment for a completed appointment, the system presents the available payment methods (Strategy pattern). The customer selects a method and confirms the amount. The PaymentService executes the transaction; if it succeeds, the appointment is marked Paid and a receipt is generated. If it fails, the customer is prompted to retry or choose an alternative payment method.


Start
Employee opens calendar
Select scheduled appointment
Set status completed
System computes final fee
Customer chooses Make Payment
Payment Service processes transaction
Payment success?
No
Show error and allow retry
Yes
Show error and allow retry
Yes
Generate receipt
End

7. Overall System Design Description

The CenGIRL Beauty Salon Management System is designed around a four-layer architecture that strictly enforces separation of concerns. The Presentation Layer interacts with three distinct actor types — Customer, Employee, and Manager — directing each request through role-based access controls implemented in the Application Layer.

The Application Layer acts as the orchestration hub. The AppointmentService coordinates the complete appointment lifecycle: creation, status updates, and cancellation. The PaymentService processes both customer-facing and internal (salary) financial transactions using the Strategy design pattern, which allows payment methods to be added or changed without modifying the core service logic. The NotificationService operates as an observer, automatically dispatching messages whenever significant appointment events occur.

All business rules and domain invariants are enforced exclusively within the Domain Layer. The Appointment entity is the central domain object; its state machine (Scheduled → Completed/Cancelled/NoShow → Paid) is protected through strict encapsulation, preventing any layer above from illegally mutating state without following the defined transition rules. Similarly, the Calendar value object associated with each Employee is exclusively managed by the AppointmentService, guaranteeing that no double-booking can occur.

The Infrastructure Layer backs all domain repositories with HashMap-based in-memory stores, ensuring consistent O(1) access for appointment lookups by ID and O(n) for filtered queries. All repositories are accessed through interfaces, making it straightforward to substitute persistent database implementations in a future phase.

Overall, the design achieves high cohesion within each layer and low coupling between them. The consistent application of SOLID principles — particularly the Single Responsibility, Open-Closed, and Dependency Inversion Principles — ensures that the system can accommodate new requirements (e.g., new service types, payment gateways, or notification channels) with minimal impact on existing code.

8. Conclusion

This Phase 2 Report has transformed the requirements gathered in Phase 1 into a comprehensive, layered design for the CenGIRL Beauty Salon Management System. The class diagram establishes a clean domain model; the sequence diagrams detail the message flows for all critical use cases; the statechart diagrams capture the lifecycle of the most state-intensive domain objects; and the activity diagrams illustrate the major process flows with their decision branches.

The design decisions taken in this phase — layered architecture, SOLID principles, and targeted use of design patterns — collectively ensure that the system will be maintainable, extensible, and robust as implementation begins in the subsequent phases.